Lab 1: Linux, C++ and Git Toolchain and RandomVector Experiment Report

Boyang Xie
Student ID: 24020036030 Course/Class: F29XS

***Abstract*— MIT VNAV Lab 1 introduces the Linux shell, Git, C++ fundamentals, and CMake. The repository contains the required text-processing results, five appended `fortune` outputs, answers to the C++ warm-up questions, the completed `RandomVector` class, and a small CMake project. On Ubuntu 24.04.5, GCC 13.3.0 compiled the C++ code without warnings and CMake 3.28.3 built the example project successfully. GNU `wc` returned 19,567 lines, 97,676 words, and 14,338 nonblank lines, matching the submitted answers. For a 20-element vector generated with seed 314159, `RandomVector` reported a mean of 0.55804, a minimum of 0.0278694, and a maximum of 0.982655. The CMake executable printed `Hello world!` and `In library`. Tests with the same files on macOS gave different word counts and pseudo-random values because the relevant system implementations differ.**

I. Objectives

MIT VNAV Lab 1 requires students to configure an Ubuntu 20.04 toolchain and practice basic shell commands, Git version control, C++ language features, and the CMake build process[1], [3]. The graded deliverables are the Git repository (5 points), shell exercises (35 points), C++ warm-up questions (20 points), and `RandomVector` (40 points)[2]. The supplementary course note was also used to check how the exercise materials are organized[4].

The report follows the same four-part structure. The shell review checks whether the recorded values can be reproduced from the source text. The C++ review examines the required interfaces and boundary cases, while the CMake review confirms the dependency between the library and executable. Git evidence is used to confirm that the deliverables were committed. This structure separates the existence of a file from evidence that its program runs successfully.

II. Materials and Methods

A. Materials and Scope of Evidence

This report uses the existing experiment files, source code, build configuration, and program output as evidence. The review consists of text counting, source inspection, independent compilation, and program execution. These operations were kept separate from the submitted files. Only results supported directly by a file or command output are reported; missing steps are not reconstructed.

B. Verification Procedure

Figure 1 shows the path from the available files to the reported conclusions. The four evidence categories were checked separately and compared with the submitted records. Compilation used temporary directories so that generated files would not alter the repository. Each numerical result can be traced to an input file, command, or program output.

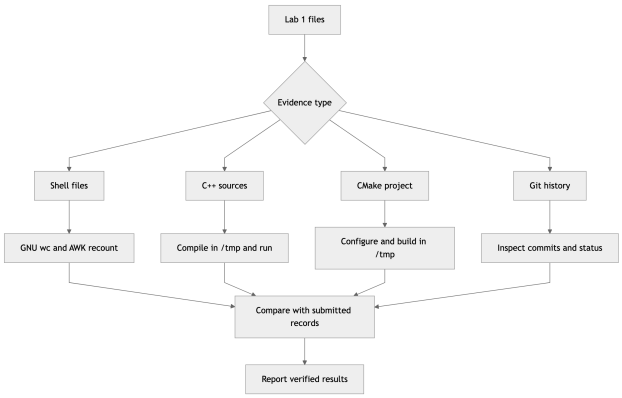


Fig. 1. Lab 1 verification procedure. Build artifacts were written to temporary directories.

C. Shell Verification

The following commands were used to count the contents of `dante.txt`:

```
wc -l -w dante.txt
awk 'NF {n++} END {print n}' dante.txt
```

`wc -l` counts newline-terminated lines, while `wc -w` counts words according to the tool and locale rules. The AWK condition `NF` selects only lines containing at least one field, so blank lines do not increase `n`. The three outputs therefore give the total line count, word count, and nonblank line count.

`fortunes.txt` contains five separate `fortune` outputs, as required. Their presence also confirms that append redirection was used instead of overwriting earlier entries.

D. C++ Implementation and Statistics

`RandomVector` stores its data in a private `std::vector<double>`. The constructor first clamps a negative length or upper bound to zero, then generates n values in $[0, m]$ using `rand()/RAND_MAX`. The `print()` method writes the elements separated by spaces. For a vector $x_1, \ldots, x_n$, the mean is

$$\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i. \tag{1}$$

All three statistical functions return 0 for an empty vector. For a nonempty vector, the minimum and maximum

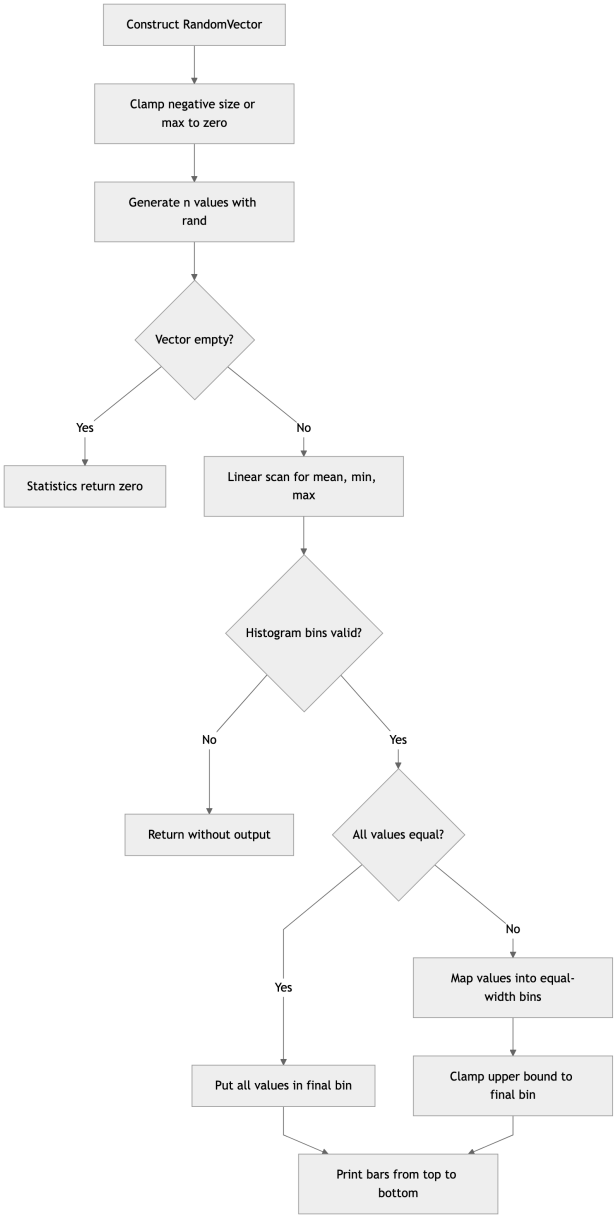


Fig. 2. RandomVector generation, statistics, and histogram output.

are found by linear scans without using the prohibited `<algorithm>` functions. The histogram maps values into b equal-width bins. For element i, the bin index is

$$k_i = \min\left(b-1, \left\lfloor \frac{x_i - x_{\min}}{x_{\max} - x_{\min}} b \right\rfloor\right). \tag{2}$$

The upper clamp places the maximum value in the final bin. If every element is equal, the denominator is zero, so the program places all elements directly in the final bin. A nonpositive bin count or an empty vector produces no histogram. Figure 2 summarizes these control branches.

The mean and extrema each require $O(n)$ time. Histogram counting takes $O(n)$ time and $O(b)$ additional space; the cost of printing also depends on the height of the largest bin. The test program uses the fixed seed 314159 so that its output

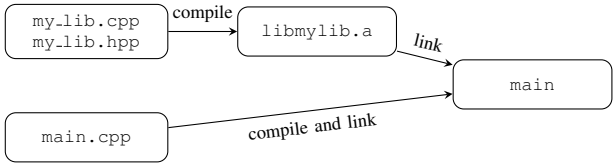


Fig. 3. CMake target dependencies observed in the source configuration and build log.

TABLE I
OFFICIAL REQUIREMENTS AND REPOSITORY EVIDENCE

Item	Evidence	Result
Git	Commit history; Lab 1 files in `15f3843`	Verified
Shell 1	`dante.txt`, `exercise1.txt`	GNU counts match
Shell 2	`fortunes.txt`, five outputs	Verified
C++ basics	Ten answers in `cpp_exercises_answers.md`	Verified
Random Vector	All six required interfaces	Compiled
CMake	Four required source files	Built and executed

is reproducible with the same C library implementation. The verification command was:

```
g++ -std=c++11 -Wall -pedantic -o random_vector \
  main.cpp random_vector.cpp
./random_vector
```

E. CMake Project

In `lab1_demo/CMakeLists.txt`, `add_library` defines `mylib` from `my_lib.cpp` and its header, and `add_executable` defines `main`. The command `target_link_libraries(main PRIVATE mylib)` links the library to `main` without propagating the dependency to other targets. This matches the official example[3].

The project was verified with separate source and build directories. The test host used CMake 3.28.3. Both `cmake -S` and `cmake --build` returned exit status 0. The resulting executable printed `Hello world!` and `In library`. These lines originate from the main program and the library function, respectively, and confirm that the link was successful. Figure 3 distinguishes compilation of the library from compilation of the program that calls it. The build log records the static library before the executable, consistent with this dependency.

III. RESULTS

A. Deliverable Coverage

Table I lists the deliverables, evidence, and verification status in the order used by the official assignment. "Verified" means that the file contents match the corresponding requirement. "Compiled" and "Built and executed" also include evidence from actual execution.

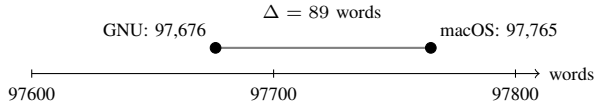


Fig. 4. Word counts for the same input file. The horizontal axis starts at 97,600.

TABLE II
RECORDED AND VERIFIED GNU/LINUX COUNTS FOR `DANTE.TXT`

Metric	Recorded	Verified	Difference
Total lines	19,567	19,567	0
Words	97,676	97,676	0
Nonblank lines	14,338	14,338	0

B. Shell Results

GNU `wc` reproduced the three values recorded in `exercise1.txt`, as shown in Table II. All three differences are zero, so the recorded values can be regenerated from the current input. The nonblank-line count is 14,338 out of 19,567 total lines; subtraction leaves 5,229 lines without fields under the AWK test. This is a derived count, not an additional shell measurement.

The same `dante.txt` file, with SHA-256 prefix and suffix `1cd578e1...c279`, produced 97,765 words on macOS and 97,676 words on GNU/Linux. The identical file hash rules out different input contents. Figure 4 isolates the 89-word difference, about 0.091% of the GNU count. Its horizontal axis is truncated to make that small difference visible; the labels give the full counts.

C. RandomVector Results

The program was compiled with GCC 13.3.0 on Ubuntu 24.04.5. The C++11 build enabled `-Wall -pedantic` and produced no compiler warnings. Compilation and execution both returned status 0. The program generated 20 values, with statistics shown in Table III. The mean lies between the minimum and maximum, and both the element count and histogram bin count match the arguments in the test program. Figure 5 plots the 20 values in their printed order. The minimum is the sixth value and the maximum is the seventeenth; the dashed line marks the reported mean. The displayed values span 0.9547856, and ten lie above the reported mean while ten lie below it. These quantities were calculated from the 20 printed values.

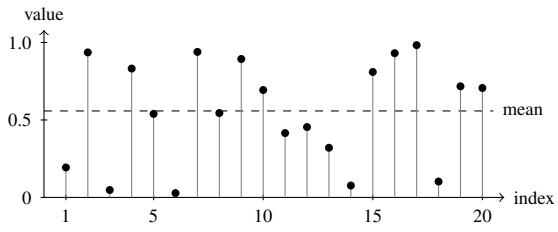


Fig. 5. The 20 printed RandomVector values in output order; the dashed line is the reported mean, 0.55804.

TABLE III
VERIFIED RESULTS FOR `RANDOMVECTOR(20)`

Metric	Result
Elements	20
Random seed	314159
Mean	0.55804
Minimum	0.0278694
Maximum	0.982655
Histogram bins	5

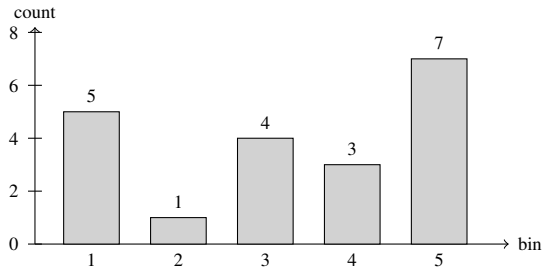


Fig. 6. Five-bin frequency analysis reconstructed from the recorded RandomVector output ($n = 20$).

The corresponding text histogram is:

```
                ***
                ***
***             ***
***      ***    ***
***      *** *** ***
***      *** *** ***
*** *** *** *** ***
```

The five bin counts reconstructed from that output are 5, 1, 4, 3, and 7. They sum to 20, matching the generated element count. The final bin contains 35% of the sample, while the second bin contains one value (5%). Figure 6 displays these counts with a numeric vertical axis, which the terminal histogram does not provide. The bin width calculated from the displayed extrema is approximately 0.19096; the final interval includes the maximum.

```
$ grep '^PRETTY_NAME=' /etc/os-release
PRETTY_NAME="Ubuntu 24.04.5 LTS"
$ g++ --version | head -n 1
g++ (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
$ cmake --version | head -n 1
cmake version 3.28.3
$ wc -l -w dante.txt
 19567  97676 dante.txt
$ awk 'NF {n++} END {print n}' dante.txt
14338
$ /tmp/lab1-report-randomvector/random_vector
Mean: 0.55804
Min: 0.0278694
Max: 0.982655
$ cmake --build /tmp/lab1-report-cmake
[100%] Built target main
$ /tmp/lab1-report-cmake/main
Hello world!
In library
```

IV. DISCUSSION

`RandomVector` implements all six required interfaces. Its statistical functions use explicit loops, satisfying the restriction against obtaining extrema directly through `<algorithm>`. Boundary handling is located in the constructor, the statistical functions, and the entry to `printHistogram()`. Negative lengths, negative upper bounds, empty vectors, and nonpositive bin counts therefore

have defined behavior. A separate branch for identical values prevents division by zero during histogram construction.

The histogram first counts the values in each bin and then prints rows from the highest level downward. The plotted counts and terminal output agree bin by bin. The chart also makes it possible to check that no generated value was lost during binning: $5 + 1 + 4 + 3 + 7 = 20$. The high count in the final bin describes this one 20-value sample; it is not evidence of a population-level distribution.

Platform differences appeared in both the shell counts and pseudorandom values. GNU and BSD toolchains returned different word counts for the same `dante.txt`, while glibc and macOS libc generated different `rand()` sequences from the same source and seed. Final numbers alone cannot explain these differences, so a complete verification record should also include the input hash, operating system, and tool versions.

V. Limitations

This report verifies the available files and newly generated output; it cannot reconstruct every action taken when the experiment was first completed. Compilation and execution show only that the current materials pass the reported checks. They do not cover untested environments, inputs, or boundary conditions. Version history shows that files were committed, but it cannot independently establish the state, access permissions, or ownership of a remote repository.

VI. Conclusion

The required shell exercise, C++ warm-up answers, `RandomVector` implementation, and CMake files are present. The GNU shell counts on Ubuntu 24.04.5 match the submitted record, and both `RandomVector` and the CMake example compiled and executed successfully.

References

[1] MIT SPARK Lab, "VNAV 2023 Lab 1," 2023. [Online]. Available: `https://vnav.mit.edu/labs_2023/lab1/`

[2] MIT SPARK Lab, "VNAV 2023 Lab 1: Exercises," 2023. [Online]. Available: `https://vnav.mit.edu/labs_2023/lab1/exercises.html`

[3] MIT SPARK Lab, "VNAV 2023 Lab 1: Build with CMake," 2023. [Online]. Available: `https://vnav.mit.edu/labs_2023/lab1/cmake.html`

[4] "Lab 1: Linux, C++, Git," course note. [Online]. Available: Notion course note